

■ AQUAGUARD

Community Water Quality & Usage Monitor

Hack-AI-Thon 2026 | SDG 6 + SDG 12

PROJECT DOCUMENTATION

1. Project Overview

AquaGuard is a Python-based water monitoring system designed for households, schools, and communities. It records daily water consumption and basic water-quality observations such as pH and turbidity. The system checks these values against project-defined demonstration thresholds, detects abnormal conditions, generates alerts and recommendations, calculates consumption trends, and produces a monitoring report.

The project is primarily implemented in Python and demonstrates Python fundamentals and Object-Oriented Programming (OOP), as required by the Hack-AI-Thon problem statement.

2. Problem Statement

Water wastage and poor awareness of water-quality conditions can create problems for households, schools, and communities. Excessive consumption may indicate wastage or leaks, while unusual pH or turbidity readings may indicate that a water sample needs further attention.

AquaGuard provides a simple way to record these observations and convert them into useful information such as quality status, alerts, trends, and conservation recommendations.

3. SDG Alignment

SDG 6 – Clean Water and Sanitation: AquaGuard supports SDG 6 by monitoring basic water-quality observations and flagging potentially unsafe sample conditions based on pH and turbidity.

SDG 12 – Responsible Consumption and Production: AquaGuard supports SDG 12 by monitoring daily water consumption, identifying excessive use, and encouraging responsible water use and conservation.

4. Objectives

- Record daily water consumption, pH, and turbidity.
- Represent monitored locations and users using OOP classes.
- Identify abnormal readings using selected project thresholds.
- Generate understandable alerts for abnormal conditions.
- Calculate a simple consumption trend from recorded readings.
- Provide practical water-conservation recommendations.
- Generate a location-wise monitoring report.

5. Scope of the Prototype

The current prototype focuses on a simple console-based Python implementation. Input is represented by sample data inside the Python program; the demonstration monitors one location; the report is displayed in the console; and no

database, IoT sensor, web API, login system, GUI, SMS/email service, or cloud service is required by the current problem statement.

6. System Design

Location: stores the place being monitored.

User: stores the person recording the readings.

WaterReading: stores one day's consumption and water-quality data and checks abnormal values.

MonitoringReport: collects readings and generates trends, alerts, recommendations, and the final report.

A Location represents where monitoring takes place. A User represents who records the data. Each WaterReading belongs to a Location and is recorded by a User. A MonitoringReport belongs to a Location and contains multiple WaterReading objects.

7. OOP Concepts Used

Classes and Objects: The project creates objects from Location, User, WaterReading, and MonitoringReport.

Encapsulation: Data and related operations are grouped inside classes.

Methods: Methods such as `is_abnormal()`, `get_alerts()`, `quality_status()`, `average_consumption()`, and `consumption_trend()` perform specific tasks.

Composition / Object Relationships: MonitoringReport stores WaterReading objects, while each reading refers to a Location and User.

Docstrings: Important classes and methods include documentation strings.

8. Data and Project-Defined Thresholds

The problem statement requires defined thresholds but does not provide numerical threshold values. Therefore, the following values are selected by the project for demonstration.

Parameter	Selected Demonstration Threshold
Minimum pH	6.5
Maximum pH	8.5
Maximum turbidity	5.0 NTU
Maximum consumption	500 L/day

Note: These are project-defined demonstration values, not numerical thresholds supplied by the Hack-AI-Thon problem statement.

9. Algorithms and Logic

Abnormal detection: Check pH against 6.5–8.5, turbidity against 5.0 NTU, and consumption against 500 L/day. If any condition is exceeded, generate the relevant alert.

Water quality: The prototype marks a reading Unsafe when pH is outside the selected range or turbidity is above the selected limit. High consumption creates a usage alert but does not itself make the water quality Unsafe.

Average: total consumption ÷ number of readings.

Trend: compare first and last readings; more than 15% higher = Increasing, more than 15% lower = Decreasing, otherwise Stable; fewer than two readings = Not enough data.

10. Input–Output Flow

1. Create Location. 2. Create User. 3. Create WaterReading objects. 4. Add readings to MonitoringReport. 5. Check readings. 6. Generate alerts and quality status. 7. Calculate average and trend. 8. Generate recommendations. 9. Print the complete report.

11. Sample Test Data

2026-09-10 — 320 L — pH 7.1 — 2.3 NTU — Safe

2026-09-11 — 450 L — pH 6.2 — 6.8 NTU — Unsafe

2026-09-12 — 510 L — pH 7.4 — 3.1 NTU — Safe

12. Expected Demonstration Result

Average consumption: approximately 426.7 L/day. Consumption trend: Increasing (possible wastage). Overall status: ATTENTION NEEDED. Alerts include low pH, high turbidity, and high consumption. Recommendations include reducing use/checking leaks, investigating rising consumption, considering filtration or further testing for high turbidity, and getting abnormal pH readings tested.

13. Testing

Normal reading: 320 L/day, pH 7.1, turbidity 2.3 NTU → Safe.

Unsafe quality: 450 L/day, pH 6.2, turbidity 6.8 NTU → Unsafe with low-pH and high-turbidity alerts.

Excessive consumption: 510 L/day, pH 7.4, turbidity 3.1 NTU → Safe quality but high-consumption alert.

Trend: 320 → 450 → 510 L → Increasing.

14. Limitations

The prototype uses sample data rather than live sensors; thresholds are project-defined; trend analysis is a simple first-versus-last comparison; the report is console-based; data is not permanently stored; and alerts are indications for attention rather than laboratory certification of drinking-water safety.

15. Future Scope

Real water-quality sensors; database storage; multiple locations; graphical dashboards; automatic data import; PDF/CSV export; and optional notification systems.

16. Requirements and Execution

Environment: Python 3.x. The prototype uses standard libraries such as datetime and typing, so no external package or API is required.

Run command: `python aquaguard.py`. If the source file has another name, run Python followed by that filename.

17. Conclusion

AquaGuard is a simple but functional Python prototype that directly addresses the Hack-AI-Thon challenge. It demonstrates the required classes, records water-use and quality observations, detects abnormal conditions, generates alerts and recommendations, calculates trends, and produces a location-wise monitoring report.

The project connects the technical solution to SDG 6 through water-quality awareness and SDG 12 through responsible water consumption and conservation.

18. Reference to the Hack-AI-Thon Requirement

The official project prompt requires a Python-based water monitoring system for households, schools, or communities, with Location/User, WaterReading, and MonitoringReport classes; recording of consumption, pH, and turbidity; abnormal-reading detection; alerts/recommendations; and location-wise or household-wise summaries. It also specifies documentation covering the problem statement, SDG mapping, objectives, system design, classes/OOP concepts, algorithms/logic, input-output flow, testing, limitations, and future scope.